

Docker Notes

Beginner to Advanced

What you will learn in these notes :

- ★ What is Docker and Why it is used
- ★ Docker Architecture
- ★ Docker Image vs Container
- ★ Dockerfile and Build Process
- ★ Important Docker Commands
- ★ Docker Volumes, Networks & Environment Variables
- ★ Docker Compose
- ★ Docker in Automation Testing
- ★ Top Docker Interview Questions

Who is this for?

- QA Engineers
- Automation Testers
- SDETs
- DevOps Beginners
- Anyone who wants to learn Docker easily!

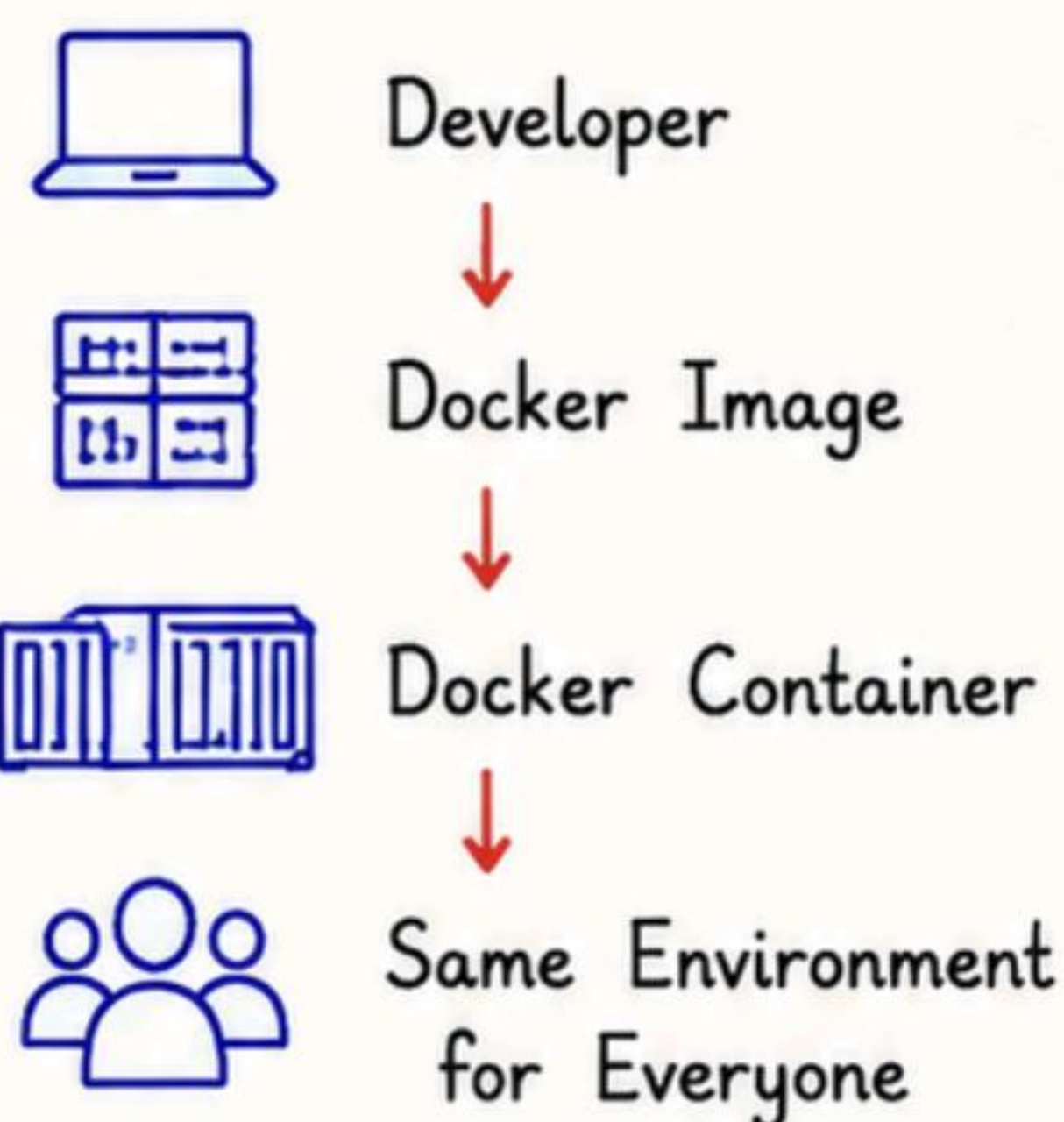


What is Docker?

Docker is a platform that helps you build, ship and run applications in **containers**.

It packages your application and all its dependencies together, so it runs **consistently** on **any environment**.

QA Insights Example



What is Docker?

Definition :

Docker is an open-source platform that helps you build, ship and run applications in **containers**.

Why Docker?

- ★ It packages application and all dependencies together.
- ★ It runs consistently on any environment.
- ★ It is lightweight, portable and fast.
- ★ It works the same on Dev, QA and Production.

Benefits :

- ★ Consistent environment
- ★ Easy deployment
- ★ Resource efficient
- ★ Isolation & Security
- ★ Scalability
- ★ Saves time and effort

QA Insights Example :

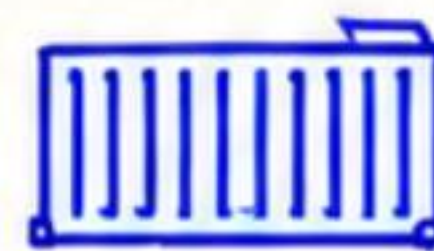
A Selenium automation project with Java, Maven, Chrome, dependencies, etc. can be packaged in a Docker container.



Developer creates the application



Docker Image is created



Docker Container is created



Same environment for everyone

Docker vs Virtual Machine

Virtual Machine

- Each VM has OS, libraries, binaries.
- Heavyweight and slower.
- Consumes more resources.
- Boot time is long.

Docker

- Shares host OS, kernel.
- Lightweight and faster.
- Consumes fewer resources.
- Starts in seconds.

Real World QA Example :

Run your automation tests in Docker containers in QA and Production without worrying about environment differences!

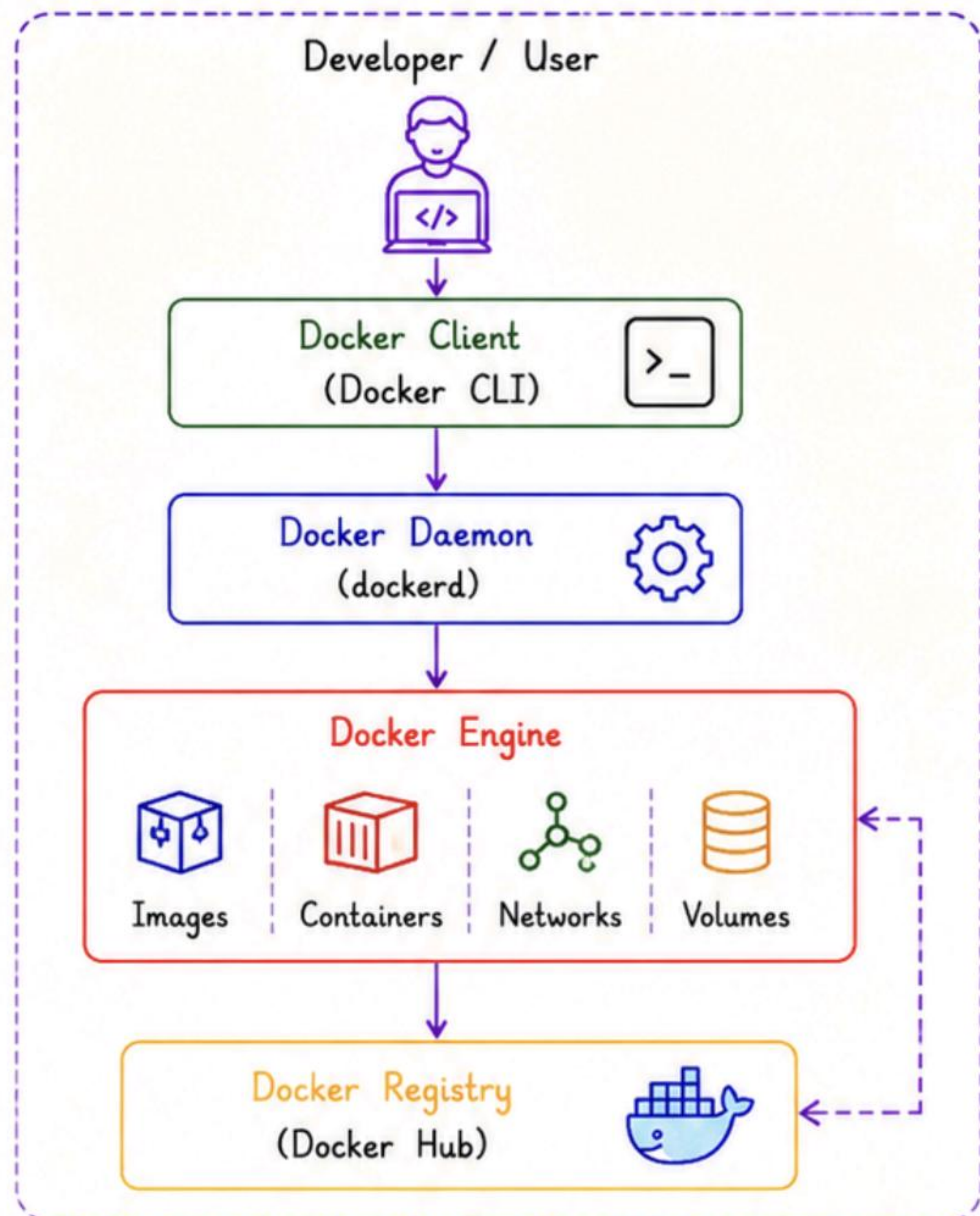


Docker Architecture

Docker uses a **client-server architecture**.

Components :

- ★ **Docker Client :**
The Docker CLI sends commands to the Docker Daemon.
- ★ **Docker Daemon :**
The heart of Docker. It builds, runs and manages Docker objects (images, containers, etc.).
- ★ **Docker Engine :**
The Docker Daemon, Images, Containers, Networks and Volumes together make up the Docker Engine.
- ★ **Docker Registry :**
Stores Docker Images. Docker Hub is the default public registry.



How it Works?

- You write a command in Docker CLI.
- The CLI sends the command to Docker Daemon.
- The Daemon builds, pulls, runs or manages Docker objects.
- Images are pulled from Registry (if not available locally).



QA Insights Example :

In automation testing, we pull Selenium standalone image from Docker Hub and run it in a container for consistent and reliable test execution.



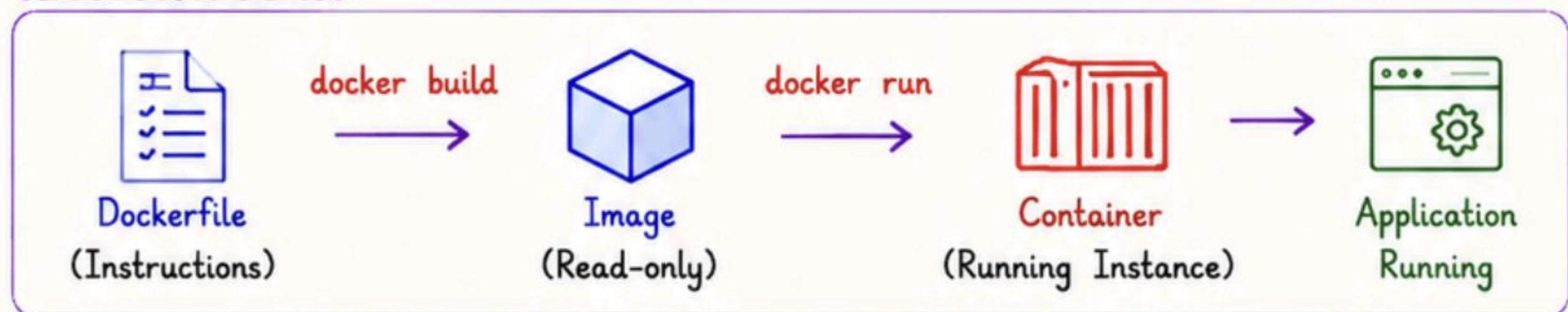
Docker Image vs Container

Understanding the difference between Image and Container is key to understanding Docker.

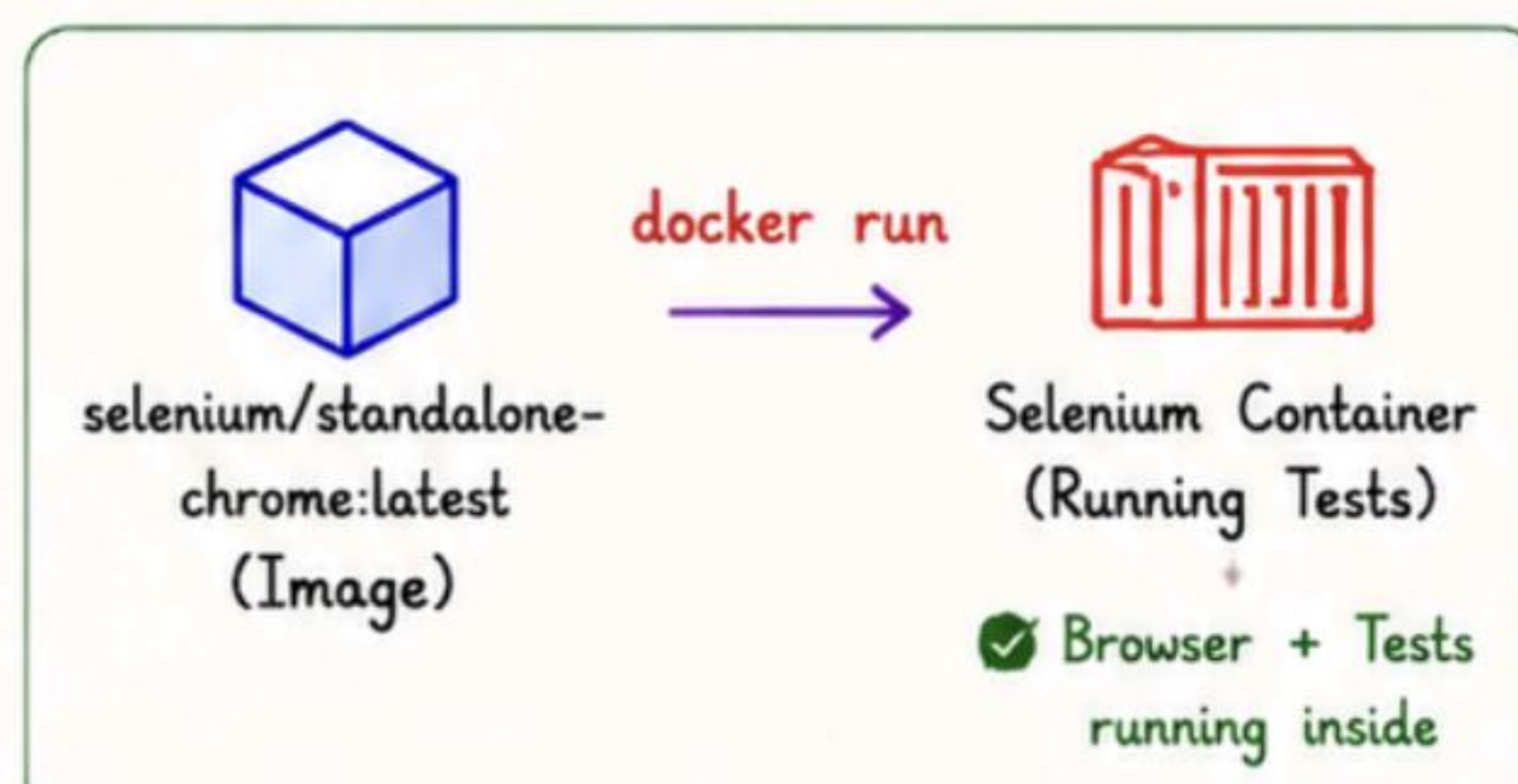
Docker Image	
★ Definition	A read-only template with instructions.
★ Nature	Immutable.
★ Purpose	Used to create containers.
★ State	Does not have state.
★ Size	Usually larger.
★ Example	selenium/standalone-chrome:latest
★ Analogy	Blueprint of a house.

Docker Container	
★ Definition	A running instance of an image.
★ Nature	Mutable.
★ Purpose	Runs the application.
★ State	Has writable layer (state).
★ Size	Usually smaller.
★ Example	Container created from selenium/standalone-chrome:latest
★ Analogy	A living house built from the blueprint.

How they relate?



QA Insights Example :



Key Takeaway :

- ★ An image is like a recipe.
- ★ A container is the running instance of that recipe.
- ★ You can create multiple containers from the same image.



Dockerfile & Build Process

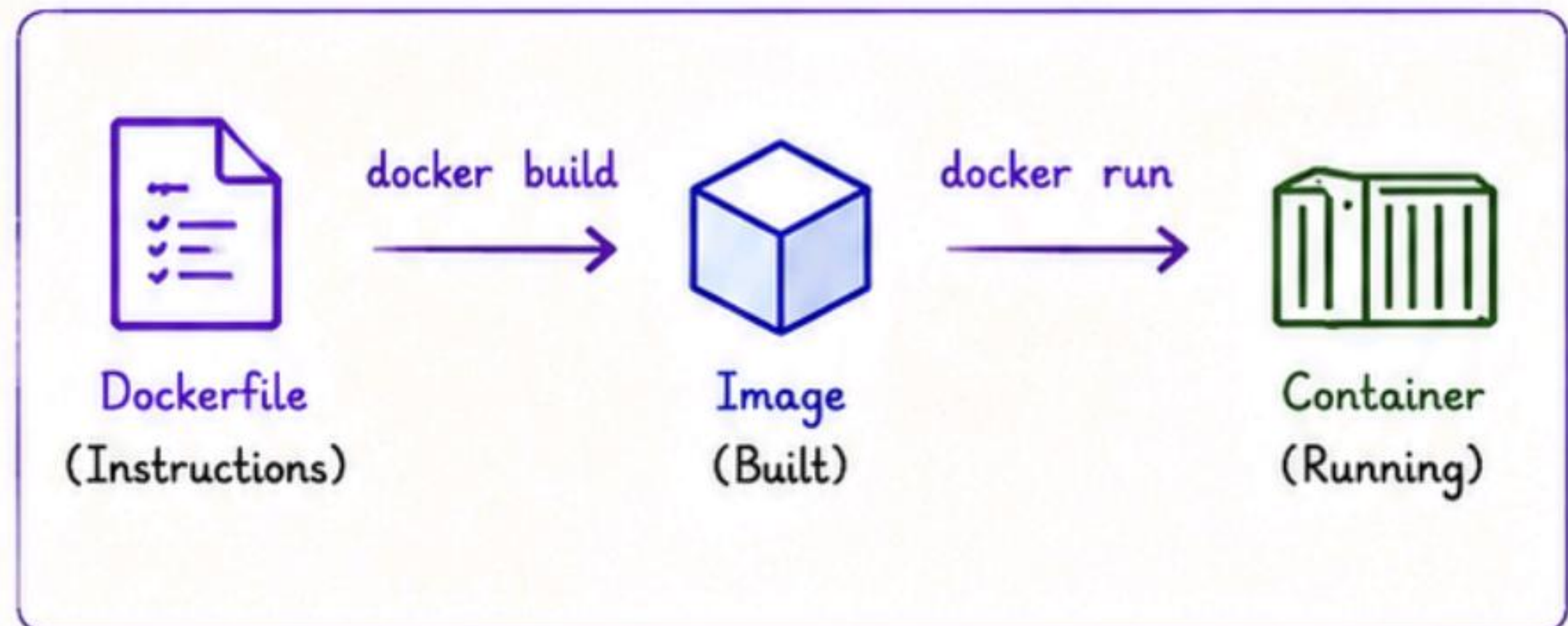
What is Dockerfile?

A Dockerfile is a text file that contains instructions to build a Docker image automatically.

Why Dockerfile?

- ★ Auto build repeatable images
- ★ Version control
- ★ Easy to share & deploy
- ★ Consistent environment

Docker Build Process



Simple Dockerfile Example

```

# Use official Python image
FROM python:3.10-slim

# Set working directory
WORKDIR /app

# Copy current directory files
COPY . /app

# Install dependencies
RUN pip install -r requirements.txt

# Expose port
EXPOSE 8000

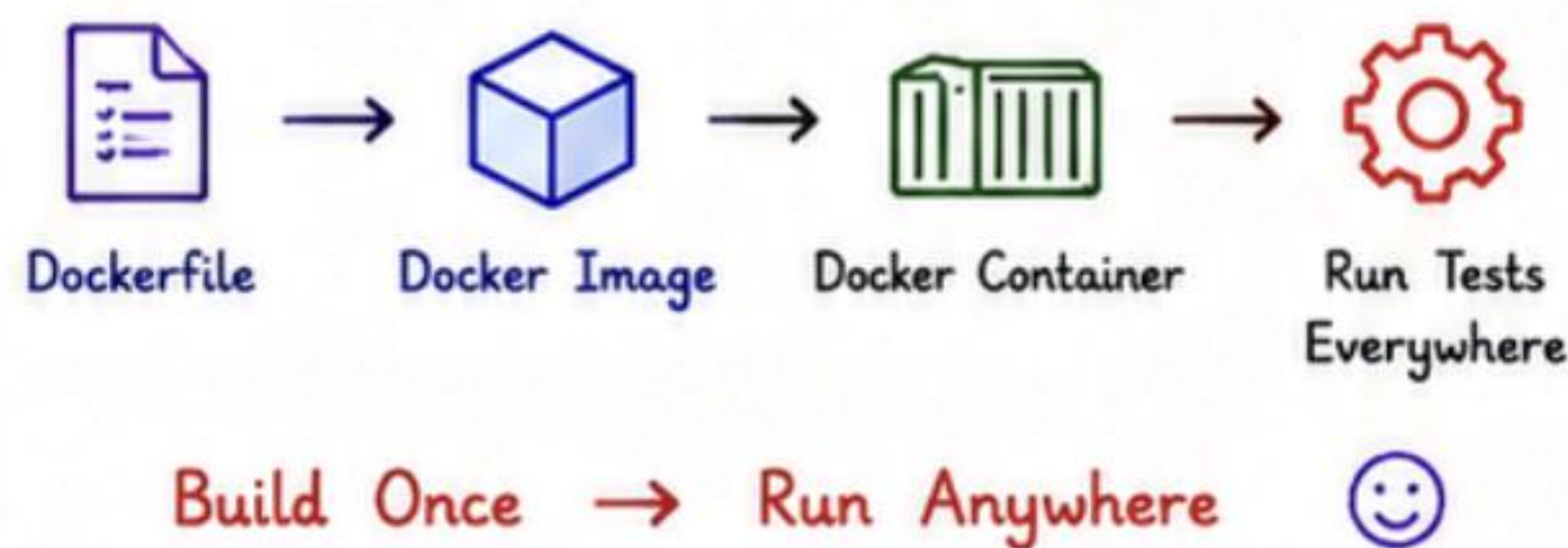
# Run application
CMD ["python", "app.py"]
  
```

Explanation

- ★ FROM : Base image
- ★ WORKDIR : Set working directory
- ★ COPY : Copy files to image
- ★ RUN : Install dependencies
- ★ EXPOSE : Expose port to access app
- ★ CMD : Command to run container

QA Insights Example

We can create a Docker image for our Selenium automation framework.



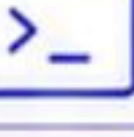
Key Takeaway

- ★ Dockerfile defines how an image is built.
- ★ Image is immutable and reusable.
- ★ Container is a running instance of an image.
- ★ Same image can run in any environment.



Most Important Docker Commands

Here are the most commonly used Docker commands every QA/Automation Engineer should know.

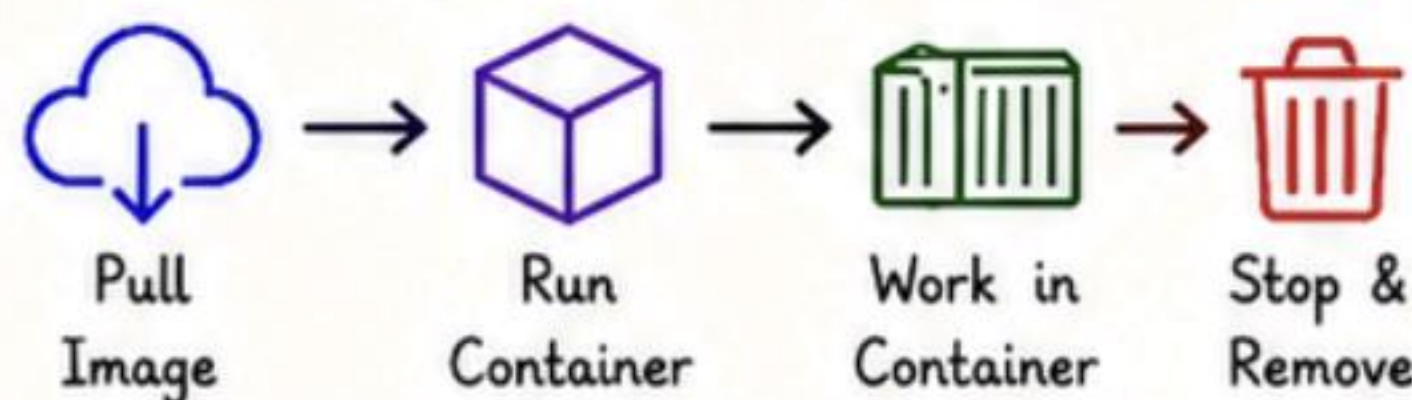
#	Command	Description	Example
1.	<code>docker --version</code>	Check Docker version	<code>docker --version</code> 
2.	<code>docker pull <image></code>	Pull image from Docker Hub	<code>docker pull python:3.10</code> 
3.	<code>docker images</code>	List all downloaded images	<code>docker images</code> 
4.	<code>docker run <image></code>	Run a container from an image	<code>docker run -d -p 8080:80 nginx</code> 
5.	<code>docker ps</code>	List running containers	<code>docker ps</code> 
6.	<code>docker ps -a</code>	List all containers (running + stopped)	<code>docker ps -a</code> 
7.	<code>docker stop <id></code>	Stop a running container	<code>docker stop a1b2c3d4e5f6</code> 
8.	<code>docker start <id></code>	Start a stopped container	<code>docker start a1b2c3d4e5f6</code> 
9.	<code>docker rm <id></code>	Remove a container	<code>docker rm a1b2c3d4e5f6</code> 
10.	<code>docker rmi <image></code>	Remove an image	<code>docker rmi python:3.10</code> 
11.	<code>docker logs <id></code>	View logs of a container	<code>docker logs a1b2c3d4e5f6</code> 
12.	<code>docker exec -it <id> bash</code>	Access container terminal	<code>docker exec -it a1b2c3d4e5f6 bash</code> 
13.	<code>docker inspect <id></code>	Inspect container details	<code>docker inspect a1b2c3d4e5f6</code> 

Quick Tip

- ★ Use tab key for auto-complete.
- ★ Add `-it` for interactive terminal.



Command Flow



- ★ These commands are the building blocks of Docker.



QA Insights Example

We use these commands to pull Selenium images, run tests in containers and maintain clean environments across QA & CI/CD.



Docker Volumes & Networks

Volumes and Networks are essential in Docker for data persistence and communication between containers.

1. Docker Volumes

- ★ Volumes are used to persist data generated by containers.
- ★ Data in a container is ephemeral. When container is removed, data is lost.
- ★ Volumes ensure data is safe and can be reused.

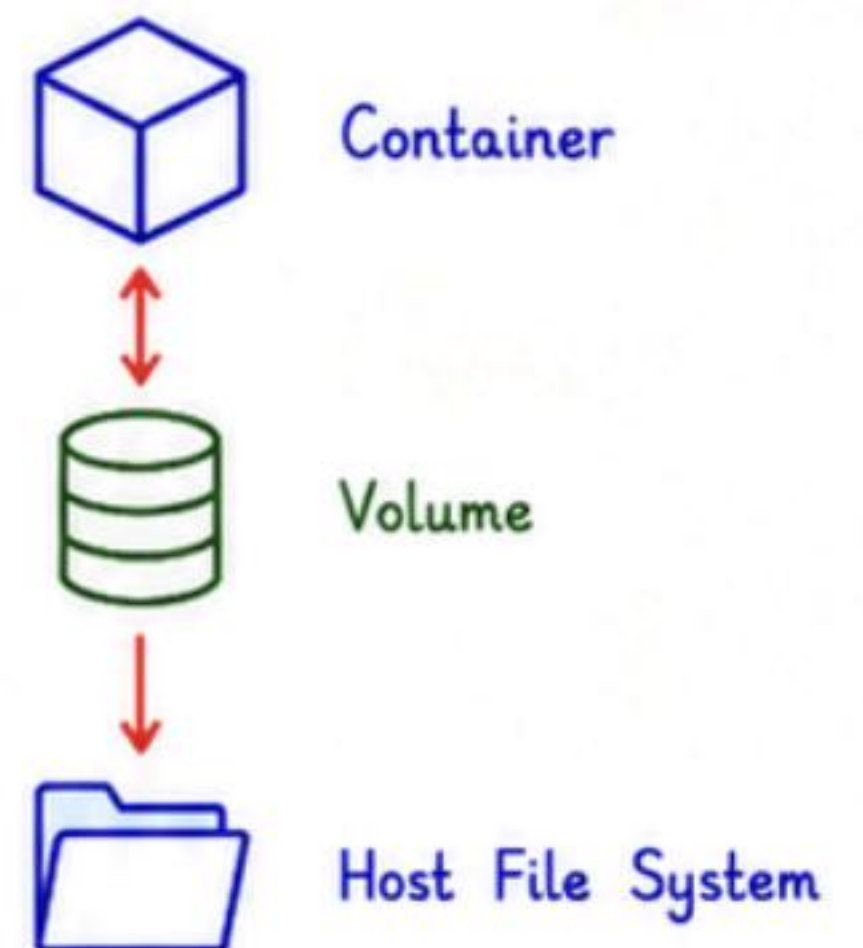
Volume Example

```
# Create volume
docker volume create mydata

# Run container with volume
docker run -d --name mycontainer \
-v mydata:/app/data nginx

# List volumes
docker volume ls
```

Volume Flow



2. Docker Networks

- ★ Networks allow containers to communicate with each other securely.
- ★ Docker provides several network drivers.
- ★ By default, containers in the same network can talk to each other using container name.

Common Network Types

Network	Description
bridge	Default network. Best for single host communication.
host	Container shares host network stack.
none	No networking.
overlay	Connects multiple Docker daemons (Swarm mode).
macvlan	Assigns MAC address to container.

Network Example

```
# Create network
docker network create mynet

# Run containers in network
docker run -d --name app1 --network mynet nginx
docker run -d --name app2 --network mynet nginx

# Containers can communicate
app1 can access app2 using
http://app2
```

3. Port Mapping



- ★ Port mapping exposes container port to host machine.

Example :

```
docker run -d -p 8080:80 nginx
```

4. Environment Variables

- ★ Used to pass configuration into containers.
- ★ Helps in building flexible applications.

Example :

```
docker run -e KEY=value nginx
```

Key Takeaway

- ★ Use Volumes for data persistence.
- ★ Use Networks for secure communication.
- ★ Use Port Mapping to expose services.
- ★ Use Environment Variables for configuration.



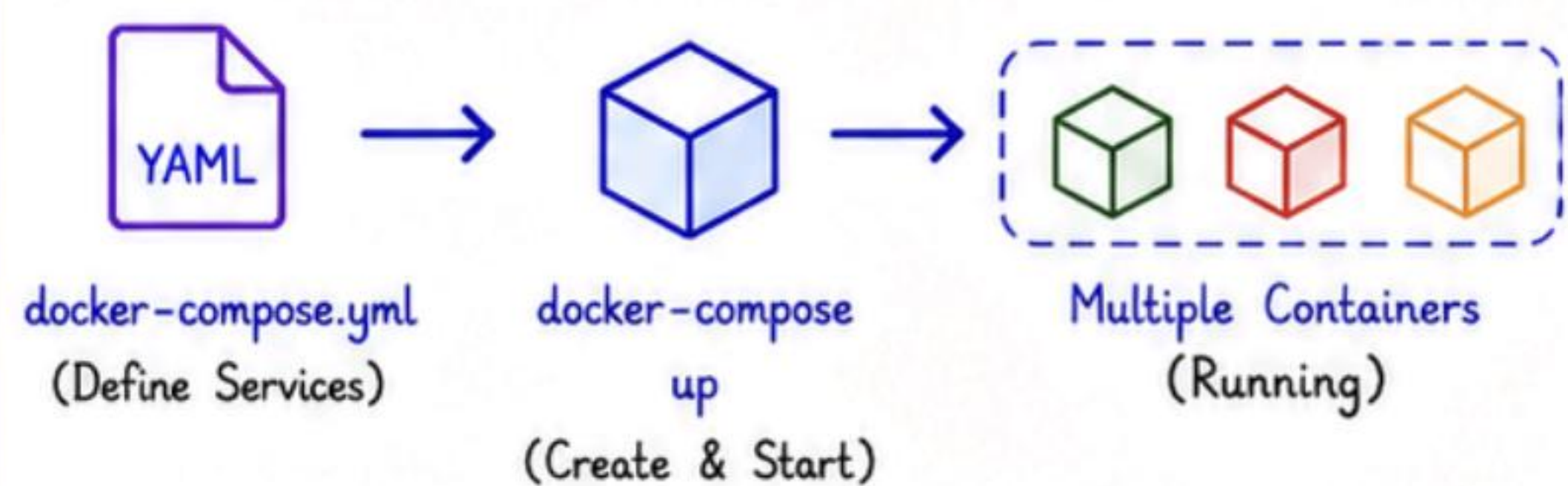
Docker Compose

Docker Compose is a tool for defining and running multi-container Docker applications.

1. What is Docker Compose?

- ★ Docker Compose allows you to define multi-container applications using a YAML file.
- ★ With a single command, you can create and start all services.
- ★ It simplifies container management for complex applications.

2. How it Works?



3. docker-compose.yml Example

```

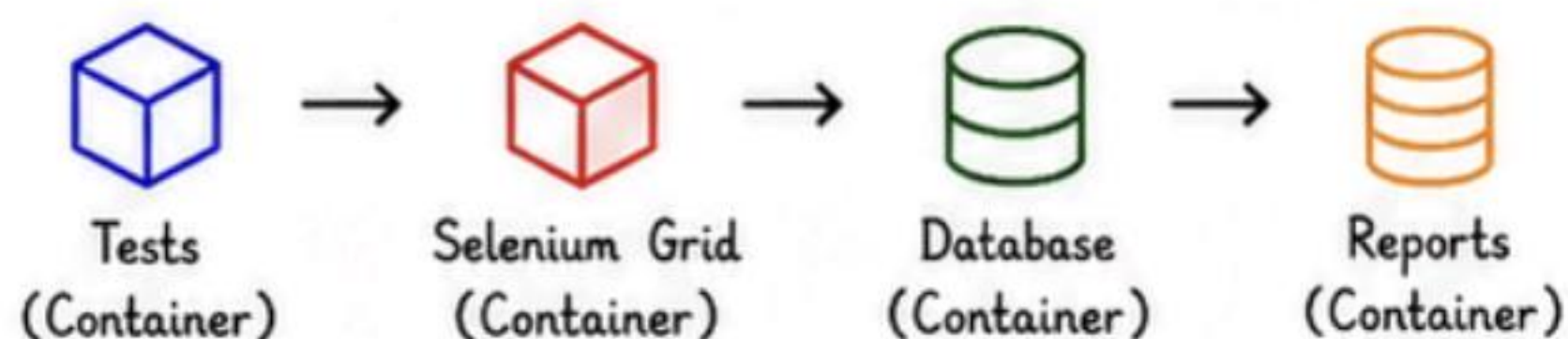
version: '3.8'  ←----- # Compose file version
services:
  web:
    image: nginx:latest  ←----- # Nginx service
    ports:
      - "8080:80"  ←----- # Map host port 8080 to container port 80
  db:
    image: mysql:5.7  ←----- # MySQL service
    environment:
      MYSQL_ROOT_PASSWORD: root  ← # Set environment variable
    volumes:
      - db_data:/var/lib/mysql  ←--- # Persist database data
  volumes:
    db_data:
  
```

4. Common Commands

Command	Description
docker-compose up	Create and start all services
docker-compose up -d	Start in detached mode (background)
docker-compose down	Stop and remove services
docker-compose ps	List running services
docker-compose logs	View logs of services
docker-compose build	Build or rebuild services
docker-compose restart	Restart services

QA Insights Example

We can use Docker Compose to run our automation stack with all required services together.



- ★ One command to start everything!

```
docker-compose up -d
```

Key Takeaway

- ★ Define services in docker-compose.yml.
- ★ Start everything with a single command.
- ★ Easy to manage multi-container applications.
- ★ Perfect for local development and CI/CD.



➤ Docker in Automation testing ⚡

Docker helps in creating **consistent**, **isolated** and **portable** environments for automation testing.

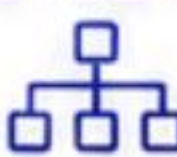
CI/CD Pipeline with Docker



Benefits in QA Automation

- ★ Consistent environment across all machines.
- ★ Easy setup and teardown of test environments.
- ★ Isolation of dependencies.
- ★ Faster execution and parallel testing.
- ★ Seamless integration with CI/CD tools.

Used With

- ★ Selenium WebDriver 
- ★ Selenium Grid 
- ★ Playwright 
- ★ API Testing (Postman, RestAssured) 
- ★ CI/CD Tools (Jenkins, GitHub Actions) 

Real World Use Cases



Key Takeaway

- ★ Docker makes automation testing **reliable**, **repeatable** and **scalable** by providing the same environment everywhere.



> Docker Troubleshooting <

Even the best setups can face issues. Here are **common problems** and how to fix them.

1. Common Issues & Solutions

Issue	Cause	Solution
 Container won't start	Wrong command or config error	Check logs using <code>docker logs <id></code>
 Port already in use	Another service using the same port	Use different port or stop the conflicting service
 Data not persisting	No volume used or wrong volume path	Use Docker volumes to persist data
 Out of Disk Space	Images, containers or volumes using too much space	Run <code>docker system prune -a</code> to clean unused data
 Network Issues	Incorrect network configuration or no access	Inspect networks using <code>docker network ls</code>

2. Quick Tips

- ★ Always check logs first.
- ★ Keep Docker and images updated.
- ★ Remove unused containers, images and volumes regularly.
- ★ Use meaningful names for containers and networks.
- ★ Monitor resource usage (CPU, Memory, Disk).



3. Useful Debug Commands

```

docker ps -a           # List all containers
docker logs <id>       # View container logs
docker inspect <id>    # Inspect container details
docker stats           # Check resource usage
docker system df       # Disk usage summary
docker system prune -a # Remove unused data
docker network ls      # List networks
docker exec -it <id> bash # Access container shell
  
```

4. Example: Fixing a Failed Container

- 1 Check status:
`docker ps -a`
- 2 View logs:
`docker logs <container_id>`
- 3 Inspect details:
`docker inspect <container_id>`
- 4 Fix issue (e.g., wrong config or port)
- 5 Restart container:
`docker start <container_id>`



Logs tell you what went wrong!

Key Takeaway

- ★ Troubleshooting is an essential skill for every QA/Automation Engineer.
- ★ Logs and inspect commands are your best friends.
- ★ Regular cleanup and monitoring prevent most issues.
- ★ A well-maintained Docker environment = Smooth automation!

